Network Programming Retake Exam: Campus Food Ordering System

Time Limit: 90 minutes
Difficulty: Medium
Topics: TCP/UDP Sockets, Concurrency (select), Protocol Design

---

Problem Statement

Create a client-server application for ordering food from campus restaurants. Students can view the menu and place orders. The system logs all orders to a separate component.

Task 1: Basic Client-Server (5 points)

Server Requirements

The server manages a restaurant menu. Each menu item has:

- **Item ID**: 4 uppercase letters (e.g., "PIZZ", "BURG")
- **Name**: String (e.g., "Margherita Pizza")
- **Price**: Integer (e.g., 1200 means $12.00)
- **Stock**: Integer (available quantity)

Hardcoded menu for Task 1:

```
menu = {
    "PIZZ": {"name": "Margherita Pizza", "price": 1200, "stock": 10},
    "BURG": {"name": "Cheeseburger", "price": 800, "stock": 15},
    "SALA": {"name": "Caesar Salad", "price": 600, "stock": 20}
}
```

Protocol

Message format: `action|item_id|quantity`
(Use `|` as separator, all fields are required)

Actions:

- `VIEW` - View item details (quantity field ignored, send "0")
- `ORDER` - Place an order for specified quantity

Example messages:

```
VIEW|PIZZ|0
ORDER|BURG|2
```

Server Responses

Situation	Response
VIEW success	`name:price:stock` (e.g., "Margherita Pizza:1200:10")
VIEW item not found	`NOT_FOUND`
ORDER success	`SUCCESS:total_price` (e.g., "SUCCESS:2400" for 2 burgers)
ORDER insufficient stock	`NO_STOCK`
ORDER item not found	`NOT_FOUND`
Invalid message format	`ERROR`

Important: When an order succeeds, decrease the stock by the ordered quantity.

Client Requirements

- Connect to server via TCP
- Read action, item_id, and quantity from **standard input**
- Support **multiple operations** in one session (loop until user types "QUIT")

- Display responses in a user-friendly format

Example session:

```
Enter action (VIEW/ORDER/QUIT): VIEW
Enter item ID: PIZZ
Enter quantity (or 0 for VIEW): 0
Sent: VIEW|PIZZ|0
Response: Margherita Pizza:1200:10
✓ Available: Margherita Pizza, Price: $12.00, Stock: 10

Enter action (VIEW/ORDER/QUIT): ORDER
Enter item ID: PIZZ
Enter quantity (or 0 for VIEW): 2
Sent: ORDER|PIZZ|2
Response: SUCCESS:2400
✓ Order placed! Total: $24.00

Enter action (VIEW/ORDER/QUIT): QUIT
Disconnecting...
```

Command-line usage:

```
python3 restaurant_server.py <ip> <port>
python3 student_client.py <ip> <port>
```

Task 2: Menu Configuration (2 points)

Objective: Load the menu from a JSON file instead of hardcoding it.

The server must accept a JSON file path as a command-line argument:

```
python3 restaurant_server.py <ip> <port> <menu.json>
```

JSON format:

```
[
  {"item_id": "PIZZ", "name": "Margherita Pizza", "price": 1200, "stock": 10},
  {"item_id": "BURG", "name": "Cheeseburger", "price": 800, "stock": 15},
  {"item_id": "SALA", "name": "Caesar Salad", "price": 600, "stock": 20}
]
```

Requirements:

- If file doesn't exist, print error and exit
- If JSON is invalid, print error and exit
- Display loaded menu at startup

Task 3: Concurrent Orders (1 point)

Objective: Allow multiple students to order simultaneously.

Requirements:

- Use `select.select()` to handle multiple TCP connections
- Server must handle at least 3 concurrent clients
- Each client operates independently (orders don't interfere)

Test: Open 3 client terminals and place orders simultaneously. All should work without blocking.

Task 4: Order Logging (2 points)

Objective: Create a UDP logger that records all successful orders.

Logger Server

Command-line:

```
python3 logger_server.py <ip> <port> <log.json>
```

Receives: UDP messages in format:

```
timestamp|student_ip|item_id|quantity|total_price
```

Example:

```
2025-12-07T15:30:00|127.0.0.1|PIZZ|2|2400
```

Writes to JSON file (append mode):

```
[
  {
    "timestamp": "2025-12-07T15:30:00",
    "student_ip": "127.0.0.1",
    "item_id": "PIZZ",
    "quantity": 2,
    "total_price": 2400
  }
]
```

After logging, send back: `ACK`

Modified Restaurant Server

Command-line:

```
python3 restaurant_server.py <ip> <port> <menu.json> <logger_ip> <logger_port>
```

Behavior:

- When an order succeeds, send notification to logger via UDP
- Wait 1 second for ACK (optional: you can ignore timeout)
- Print confirmation when ACK received

Grading

Task	Points	Description
Task 1	5	Basic TCP client-server with protocol
Task 2	2	JSON menu loading
Task 3	1	Concurrent connections (select)
Task 4	2	UDP logger component
Total	**10**	

Passing grade: 5/10 (50%)

Clarifications

Message Format Rules

- Always use `|` as separator (pipe character)
- All three fields are required: `action|item_id|quantity`
- For VIEW, set quantity to "0"
- Item IDs are case-sensitive (use UPPERCASE)

Connection Behavior

- Client must stay connected after each request
- Client only disconnects when user types "QUIT"
- Server must handle client disconnects gracefully

Stock Management

- Stock can reach 0 (sold out)

- If stock is 0, order fails with `NO_STOCK`
- Orders that exceed available stock should fail with `NO_STOCK`

Error Handling

- Invalid message format → `ERROR` response
- Item not in menu → `NOT_FOUND` response
- Client should display errors clearly

Sample Files to Submit

For full points (Tasks 1-4):

- `restaurant_server.py`
- `student_client.py`
- `logger_server.py`
- `menu.json` (sample data)

For passing (Tasks 1-2):

- `restaurant_server.py` (with Task 2)
- `student_client.py`
- `menu.json`

Testing Checklist

- [ ] Client can VIEW an item successfully
- [ ] Client can ORDER an item successfully (stock decreases)
- [ ] Server rejects orders when stock is 0
- [ ] Server rejects invalid item IDs
- [ ] Client can perform multiple operations without reconnecting
- [ ] Multiple clients can connect simultaneously (Task 3)
- [ ] Logger receives and saves orders (Task 4)

Example Run

```
# Terminal 1: Start logger
python3 logger_server.py 127.0.0.1 6000 orders.json

# Terminal 2: Start restaurant server
python3 restaurant_server.py 127.0.0.1 5000 menu.json 127.0.0.1 6000

# Terminal 3: Start client
python3 student_client.py 127.0.0.1 5000
```

Good luck! 🍕